

LAB 02. PART2: 2D Geometric Transformations

Computer Vision

2026-2027

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên cần đạt được:

- Hiểu bản chất của các **biến đổi hình học 2D** trong xử lý ảnh.
- Cài đặt và sử dụng các phép biến đổi:
 - Translation (T)
 - Rotation (R)
 - Scaling (S)
 - Skew / Shear (K)
- Hiểu cách xây dựng các phép biến đổi dựa trên **công thức toán học** sử dụng thư viện NumPy.
- Hiểu các vấn đề thực tế phát sinh khi biến đổi ảnh:
 - canvas không đủ lớn
 - xuất hiện pixel bị thiếu (pixel holes)
 - hiện tượng crop ảnh
- So sánh thuật toán tự cài đặt với các hàm tối ưu trong thư viện **OpenCV**.

2 Chuẩn bị môi trường

Các thư viện cần thiết:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
```

Hàm hiển thị ảnh:

```
def show(img, title="image"):
    plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis("off")
    plt.show()
```

Đọc ảnh từ tệp:

```
img = cv2.imread("image.jpg",0)
show(img)
```

3 Biểu diễn toán học của biến đổi hình học

Một ảnh số có thể được biểu diễn như một hàm:

$$f(x, y)$$

Một phép biến đổi hình học sẽ tạo ra ảnh mới:

$$g(x', y') = f(x, y)$$

trong đó:

$$(x', y') = T(x, y)$$

3.1 Biểu diễn bằng ma trận Affine

Trong xử lý ảnh, các phép biến đổi hình học thường được biểu diễn bằng **ma trận affine**:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Ma trận này có thể biểu diễn nhiều loại biến đổi:

- Translation
- Rotation
- Scaling
- Shear

4 Translation

4.1 Công thức

$$x' = x + t_x$$

$$y' = y + t_y$$

4.2 Cài đặt trực tiếp theo công thức

```
def translate_basic(img, tx, ty):  
  
    h, w = img.shape  
    out = np.zeros_like(img)  
  
    for y in range(h):  
        for x in range(w):  
  
            new_x = x + tx
```

```

new_y = y + ty

if 0 <= new_x < w and 0 <= new_y < h:
    out[new_y, new_x] = img[y,x]

return out

```

4.3 Inverse Mapping

Thay vì ánh xạ từ ảnh gốc sang ảnh đích, ta thực hiện ánh xạ ngược:

```

def translate_inverse(img, tx, ty):

    h,w = img.shape
    out = np.zeros((h,w), dtype=img.dtype)

    for y in range(h):
        for x in range(w):

            src_x = x - tx
            src_y = y - ty

            if 0 <= src_x < w and 0 <= src_y < h:
                out[y,x] = img[src_y,src_x]

    return out

```

4.4 Sử dụng OpenCV

```

tx = 50
ty = 30

M = np.float32([[1,0,tx],
                [0,1,ty]])

translated = cv2.warpAffine(
    img,
    M,
    (img.shape[1], img.shape[0]),
    borderValue=0)

show(translated, "Translation_OpenCV")

```

4.5 Bài tập

1. Dịch ảnh sang phải 50 pixel.
2. Dịch ảnh lên trên 30 pixel.
3. So sánh kết quả giữa:

- thuật toán tự cài đặt bằng NumPy
- hàm OpenCV

5 Scaling

5.1 Công thức

$$x' = s_x x$$

$$y' = s_y y$$

5.2 Cài đặt cơ bản

```
def scale_basic(img, sx, sy):
    h,w = img.shape
    out = np.zeros_like(img)

    for y in range(h):
        for x in range(w):

            new_x = int(x*sx)
            new_y = int(y*sy)

            if new_x < w and new_y < h:
                out[new_y,new_x] = img[y,x]

    return out
```

5.3 Điều chỉnh kích thước canvas

Kích thước ảnh mới:

$$W' = s_x W$$

$$H' = s_y H$$

```
def scale_canvas(img, sx, sy):
    h,w = img.shape

    new_w = int(w*sx)
    new_h = int(h*sy)

    out = np.zeros((new_h,new_w), dtype=img.dtype)

    for y in range(new_h):
        for x in range(new_w):
```

```

src_x = int(x/sx)
src_y = int(y/sy)

if src_x < w and src_y < h:
    out[y,x] = img[src_y,src_x]

return out

```

5.4 Scaling bằng OpenCV

```

scaled = cv2.resize(
    img,
    None,
    fx=1.5,
    fy=1.5,
    interpolation=cv2.INTER_LINEAR)

show(scaled, "Scaling_OpenCV")

```

5.5 Các phương pháp interpolation

- cv2.INTER_NEAREST
- cv2.INTER_LINEAR
- cv2.INTER_CUBIC

6 Rotation

6.1 Công thức

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

6.2 Rotation cơ bản

```

def rotate_basic(img, angle):

    h,w = img.shape
    out = np.zeros_like(img)

    rad = np.radians(angle)

    cos = np.cos(rad)
    sin = np.sin(rad)

```

```

cx = w//2
cy = h//2

for y in range(h):
    for x in range(w):

        x0 = x - cx
        y0 = y - cy

        new_x = int(cos*x0 - sin*y0 + cx)
        new_y = int(sin*x0 + cos*y0 + cy)

        if 0 <= new_x < w and 0 <= new_y < h:
            out[new_y,new_x] = img[y,x]

return out

```

6.3 Rotation bằng OpenCV

```

(h,w) = img.shape

center = (w//2, h//2)

M = cv2.getRotationMatrix2D(center,45,1.0)

rotated = cv2.warpAffine(
    img,
    M,
    (w,h),
    borderValue=0)

show(rotated, "Rotation_OpenCV")

```

7 Skew / Shear

7.1 Công thức

$$x' = x + ky$$

$$y' = y$$

7.2 Cài đặt

```

def shear_basic(img, k):

    h,w = img.shape
    out = np.zeros_like(img)

```

```

for y in range(h):
    for x in range(w):

        new_x = int(x + k*y)

        if new_x < w:
            out[y,new_x] = img[y,x]

return out

```

7.3 Shear bằng OpenCV

```

k = 0.3

M = np.float32([
    [1,k,0],
    [0,1,0]
])

new_w = int(img.shape[1] + abs(k*img.shape[0]))

sheared = cv2.warpAffine(
    img,
    M,
    (new_w, img.shape[0]),
    borderValue=0)

show(sheared, "Shear_OpenCV")

```

8 Bài tập nâng cao

1. So sánh hai pipeline:

$$R(30^\circ) \rightarrow S(1.5)$$

và

$$S(1.5) \rightarrow R(30^\circ)$$

Giải thích sự khác biệt của kết quả.

2. So sánh hai trường hợp:

$$T \rightarrow R$$

và

$$R \rightarrow T$$

3. Kết hợp nhiều phép biến đổi:

$$T(40, 20), R(45^\circ), S(0.5)$$

4. Cài đặt hàm tổng quát:

```
affine_transform(img, matrix)
```

5. So sánh tốc độ thực thi giữa:

- thuật toán tự cài đặt
- OpenCV warpAffine

9 Kết luận

Bài thực hành này giúp sinh viên hiểu:

- bản chất toán học của các phép biến đổi hình học
- sự khác biệt giữa forward mapping và inverse mapping
- cách xử lý canvas khi biến đổi ảnh
- cách sử dụng các hàm tối ưu trong OpenCV